

QueryFabric: A Portable Analytical Query Compiler for Scientific Platforms

Can H. Tartanoglu
codeberg.org/caniko/queryfabric

Abstract

Scientific data platforms increasingly span multiple storage backends, yet their query surfaces remain tightly coupled to individual engines—forcing operators to choose between portability and backend-specific performance. QueryFabric is an open-source Rust library that decouples query authoring from backend execution through a four-stage compiler pipeline: *parse*, *bind*, *analyze*, and *emit*. It provides a verified portable SQL subset, pre-emission capability analysis with structured diagnostics, provenance receipts for every compiled artifact, and a data-sovereignty layer covering GDPR-aligned access, content-addressed export bundles, and DOI minting. We describe QueryFabric’s architecture, its extensible backend adapter model (ClickHouse, PostgreSQL, and custom targets), and its role as the federation backbone for downstream projects. A production deployment—SynDB, a federated microscopy platform on the NRP Nautilus Kubernetes cluster—demonstrates real-world query latency (p50 1.9–3.6 s on a Kubernetes cluster) and a verified data-minimising federation pipeline. QueryFabric is available at <https://codeberg.org/caniko/queryfabric> under the Apache License, Version 2.0.

1 Introduction

Scientific data platforms face a fundamental tension: users need expressive, backend-independent query capabilities, while operators rely on specialised storage engines for performance. A platform that starts with PostgreSQL may later adopt ClickHouse for columnar analytics, DuckDB for embedded workloads, or a federated topology spanning multiple sites. Without a compiler boundary between query text and backend execution, each new backend duplicates parsing, validation, and emission logic—or, worse, couples the query surface to a single engine’s dialect.

QueryFabric addresses this gap with a portable analytical query compiler designed as a backbone for data federations. It sits between the query surface and the execution layer, providing a four-stage pipeline—*parse*, *bind_and_validate*, *analyze*, and *emit*—that converts text queries into typed, backend-specific artifacts with full provenance. By design, QueryFabric stops short of execution, authentication, routing, and orchestration; those responsibilities belong to the host platform. This boundary keeps the compiler small, auditable, and embeddable.

The project originated from the need to decouple a scientific query dialect (SyQL) from its ClickHouse and PostgreSQL backends in a federated microscopy data platform deployed on the NRP Nautilus Kubernetes cluster. It has since been extracted into a standalone library with 35 internal crates, a facade crate that exposes the public API, Python bindings via PyO3, and a hardened NixOS deployment module.

This paper makes the following contributions:

1. The architecture of a portable four-stage query compiler with capability analysis and provenance receipts.
2. A verified portable SQL subset with machine-readable backend capability manifests.
3. A data-sovereignty and federation toolkit for self-hosted and multi-instance deployments.

4. A production case study—SynDB—demonstrating real-world query performance and a verified federation pipeline.

2 Architecture

QueryFabric is organised around four explicit compilation stages that form a semantic boundary between raw query text and executable backend artifacts. Figure 1 shows the pipeline and its Rust API.

2.1 Parsed Query

`Dialect::parse(&str)` produces a `ParsedQuery`. Parsing handles syntax validation, source-span tracking, and dialect-specific metadata such as SyQL’s `SCOPE` and `DOWNLOAD` directives, which are preserved as opaque execution hints rather than relational operators. The generic SQL dialect uses `sqlparser` internally; SyQL and custom dialects extend the same trait.

At this stage the query is unresolved: column names, function signatures, and data types have not been validated against a catalog.

2.2 Bound Query

`bind_and_validate(parsed, catalog, params)` produces a `BoundQuery`. This is where QueryFabric commits to meaning. The catalog—a neutral trait-based interface—resolves relation names, column types, function signatures, and parameter placeholders. Coercions and result shapes are checked here; if a query fails because of schema mismatches, missing functions, or type errors, it fails at binding with structured diagnostics.

The catalog contract is deliberately generic. Hosts provide relation schemas (`RelationSchema`), function signatures, and a snapshot identifier for provenance tracking. A `MemoryCatalog` is provided for testing and small deployments; production hosts implement the trait against their own schema store.

2.3 Backend Analysis

`analyze(bound, adapter, catalog)` produces `BackendAnalysis`. Analysis is intentionally separate from emission: it lets a host ask each candidate backend whether it supports a given query before choosing where to run it.

Each backend adapter declares a `CapabilitySet`—the set of SQL features it supports—and analysis checks every capability requirement that binding computed. The result is a structured diagnostic with stable codes, severity levels, remediation hints, and backend context. The difference between “backend unsupported” and “PostgreSQL rejects this query because it requires a ClickHouse namespaced function” is a concrete engineering improvement for multi-backend hosts.

2.4 Emission

`emit(bound, adapter, catalog)` produces `EmitArtifact`. For the current release, emission produces SQL text for ClickHouse and PostgreSQL. Each artifact carries a `ProvenanceReceipt` recording the compiler version, catalog snapshot identifier, and dialect, so downstream systems can answer “where did this SQL come from?” without out-of-band metadata.

The artifact seam is kept open for non-SQL targets: the same `BoundQuery` that emits ClickHouse SQL today could emit a DataFusion logical plan or a Substrait message tomorrow, without changing the facade.

2.5 Host Boundary

A deliberate design principle (documented in the project’s decision log as D003) keeps host execution outside QueryFabric. The library does not execute queries, manage authentication, or own job orchestration. This boundary is not a gap; it is the feature that keeps the core small (~17 MB release binary, ~63 MB Nix closure), simple to audit, and trivial to embed in platforms with existing auth, routing, and scheduling layers.

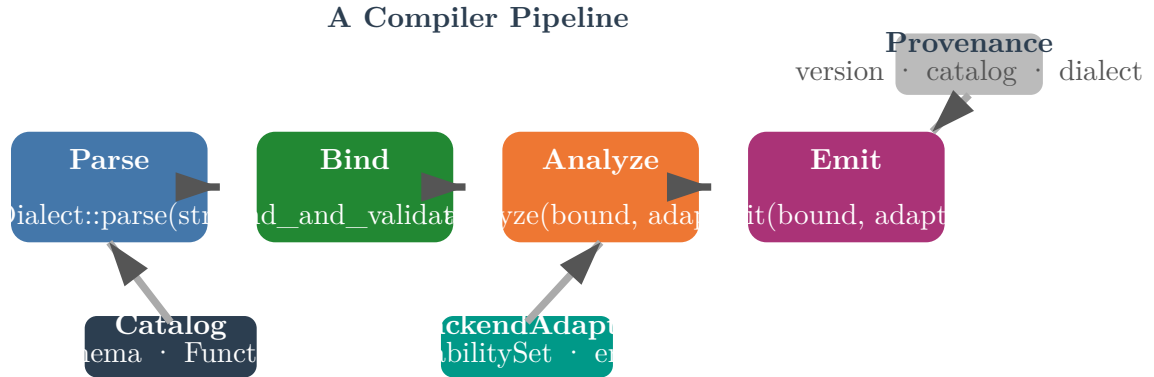


Figure 1: **Compiler pipeline architecture.** (A) The four-stage pipeline: `parse` converts dialect text into a `ParsedQuery`; `bind_and_validate` resolves names and types through a catalog into a `BoundQuery`; `analyze` checks backend support via capability manifests; `emit` produces backend-specific SQL with provenance receipts. (B) Rust API example showing the pipeline in use.

3 Portability and Conformance

Portability in QueryFabric is defined by a machine-readable capability model and a verified conformance corpus, not by what the parser happens to accept.

3.1 Verified Portable Subset

The first public release targets a documented, test-backed subset of SQL:

- `SELECT`, `WHERE`, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`, `OFFSET`, `DISTINCT`
- `CASE` expressions
- Typed positional (`$1`) and named (`:name`) parameters
- All join types: `INNER`, `LEFT`, `RIGHT`, `FULL`, `CROSS`
- Non-recursive CTEs (`WITH`)
- Derived subqueries in `FROM`
- Scalar subqueries and `IN` subqueries (where validated)
- `UNION ALL`
- Common scalar and aggregate functions through the function registry
- Window functions: `RANK`, `DENSE_RANK`, `ROW_NUMBER`, `LAG`, `LEAD`, `FIRST_VALUE`, `LAST_VALUE`

This subset is enforced by 28 conformance test cases in the portable-subset corpus, each specifying expected schemas and backend support status.

3.2 Backend Capability Manifests

Each backend adapter declares its capabilities in a machine-readable JSON manifest. Both ClickHouse and PostgreSQL currently support CTEs, derived tables, joins, window functions, set operations, aggregates, scalar and IN subqueries, limit/offset, and `EXPLAIN`. ClickHouse additionally supports namespaced functions, approximate aggregates, isolated execution via Kubernetes, and UUID-to-string conversion in Arrow output.

When a query requires a capability that a backend does not support, analysis produces a structured diagnostic before emission—catching incompatibilities at compile time rather than at execution time.

A Capability Matrix

	ClickHouse	PostgreSQL
Aggregates	[X]	[X]
CTEs	[X]	[X]
Joins	[X]	[X]
Windows	[X]	[X]
Set Ops	[X]	[X]
Subqueries	[X]	[X]
Limit/Offset	[X]	[X]
Namespaced Funcs	[X]	[]
Approx Agg	[X]	[]
Isolated Exec	[X]	[]

Figure 2: **Portability and capabilities.** (A) Capability matrix showing supported features for ClickHouse and PostgreSQL backends. (B) Conformance test results for the 28 portable-subset cases, grouped by feature category.

4 Applications as a Federation Backbone

QueryFabric’s architecture supports several integration patterns beyond simple single-backend compilation.

4.1 Python Bindings

Python bindings via PyO3 expose the same four-stage API to data scientists. The `queryfabric` Python package mirrors the Rust facade: `parse_sql` and `parse_syql` produce parsed queries; `bind_and_validate` resolves them against a `MemoryCatalog`; `analyze_clickhouse` and `analyze_postgres` check backend support; and `emit_clickhouse_sql` and `emit_postgres_sql` produce executable artifacts. This enables server-side validation of user-written SyQL before execution.

4.2 Custom Backend Adapters

The `BackendAdapter` trait defines the contract for new backends:

```
trait BackendAdapter {
  fn name(&self) -> &'static str;
  fn capabilities(&self) -> CapabilitySet;
  fn analyze(&self, query: &BoundQuery,
            catalog: &dyn Catalog) -> BackendAnalysis;
```

```

fn emit(&self, query: &BoundQuery,
       catalog: &dyn Catalog) -> Result<EmitArtifact>;
}

```

A DuckDB adapter, for example, requires roughly 50 lines of Rust: declare capabilities (aggregates, joins, CTEs, windows, limit/offset), delegate analysis to the generic capability checker, and reuse the generic SQL emitter with `SqlBackend::Other("duckdb")`. The host gains parsing, binding, capability analysis, provenance, and parameter validation for free.

4.3 Data Sovereignty Layer

Beyond compilation, QueryFabric provides a data-sovereignty toolkit for self-hosted services:

- `queryfabric-access`: GDPR Art. 15/16/17 traits for access, rectification, and erasure over generic resources.
- `queryfabric-portability`: content-addressed export bundles with BLAKE3 content hashing, JSON-LD manifests, and DOI minting via DataCite.
- `queryfabric-tenancy`: multi-tenant accounts, collections, and groups with isolation outside the compiler core.

4.4 Federation Substrate

The federation crate set (`queryfabric-cluster` and `queryfabric-federation`) provides a libp2p-based substrate for multi-node deployments. It includes hub/cluster actor models, a DHT-based node registry, health monitoring with a wire-stable vocabulary (`healthy`, `degraded`, `unreachable`), and a schema-sync protocol. The hub decomposes aggregate queries (SUM, COUNT, AVG, MIN, MAX) into scatter-gather plans: each node computes local summaries; the hub merges them using parallel algorithms (e.g., Welford’s method for variance). This enables data-minimising meta-analysis where raw data never leaves individual clusters.

4.5 NixOS Deployment

QueryFabric ships a hardened NixOS module with systemd credentials for secrets, a multi-instance option, and an end-to-end VM test. The total deployment footprint is ~17 MB for the release binary and ~63 MB for the Nix closure; idle RSS is ~2.9 MiB, and peak under load is ~14 MiB.



Figure 3: **Applications and integration patterns.** (A) Python embedding: data scientists write SyQL, the server validates it through QueryFabric’s pipeline, then emits and executes. (B) Custom adapter pattern: implement `BackendAdapter` for any backend (DuckDB shown). (C) Data sovereignty workflow: GDPR request triggers access-policy evaluation, resource collection, content-addressed bundle assembly, and optional DOI minting.

5 Case Study: SynDB

SynDB is a downstream project that uses QueryFabric as its query compiler and federation backbone. It is a federated neuroanatomical data platform running on the NRP Nautilus Kubernetes cluster with 5 shard regions across US partner institutions, ingesting connectomics datasets including Hemibrain, FlyWire, MICrONS, and MANC.

5.1 Architecture

SynDB combines PostgreSQL for metadata, user accounts, and dataset annotations with ClickHouse for columnar neuroanatomical data. QueryFabric sits as the semantic compiler boundary: SyQL—a dialect extending SQL with `SCOPE` directives for federation and `DOWNLOAD` for data export—is parsed, bound against a facility catalog, analyzed for ClickHouse and PostgreSQL support, and emitted to the appropriate backend. A single query compiles once and executes against either backend.

5.2 Live Query Performance

To characterise production query performance, a cluster-resident benchmark controller ran 160 direct-query measurements against the Hemibrain dataset on a Kubernetes cluster. Four query templates were tested across cold and warm cache states:

- **Neuron scan 1K:** p50 2,154 ms (cold), 2,079 ms (warm)
- **Primary-key lookup:** p50 1,869 ms (cold), 1,910 ms (warm)
- **Synapse scan 1K:** p50 2,043 ms (cold), 2,038 ms (warm)
- **Neuron wide 10K:** p50 2,941 ms (cold), 3,623 ms (warm)

p95 latencies ranged from 2,024 ms (primary-key lookup, cold) to 12,329 ms (neuron wide 10K, warm). All analytics API endpoints backed by materialised views achieved sub-10 ms p50 latency.

5.3 Federated Meta-Analysis

SynDB’s federation pipeline — built on QueryFabric’s federation substrate — implements a two-phase data-minimising meta-analysis. Phase 1 computes summary statistics (N , mean, std) locally at each cluster via ClickHouse’s `remoteSecure()`. Phase 2 combines these summaries at the hub using Welford’s parallel variance algorithm, computing effect sizes (Cohen’s d , Hedges’ g) and heterogeneity metrics (Cochran’s Q , I^2) without transferring individual-level data. Validation against centralised production-cache partitions confirmed that summary-only and full-data estimates agree to numerical precision.

6 Conclusion

QueryFabric is a portable analytical query compiler that decouples query authoring from backend execution through a four-stage pipeline with capability analysis, provenance receipts, and a data-sovereignty toolkit. Its verified portable SQL subset, extensible backend adapter model, and federation substrate make it suitable as a backbone for multi-backend and multi-instance scientific data platforms.

The project is open-source under the Apache License, Version 2.0, at <https://codeberg.org/caniko/queryfabric>. Future work includes import-side portability (ingesting content-addressed bundles into differently-shaped catalogs), embedded backends (DataFusion and SQLite), and federation hardening (hub failover, NAT traversal, schema-sync conflict resolution).

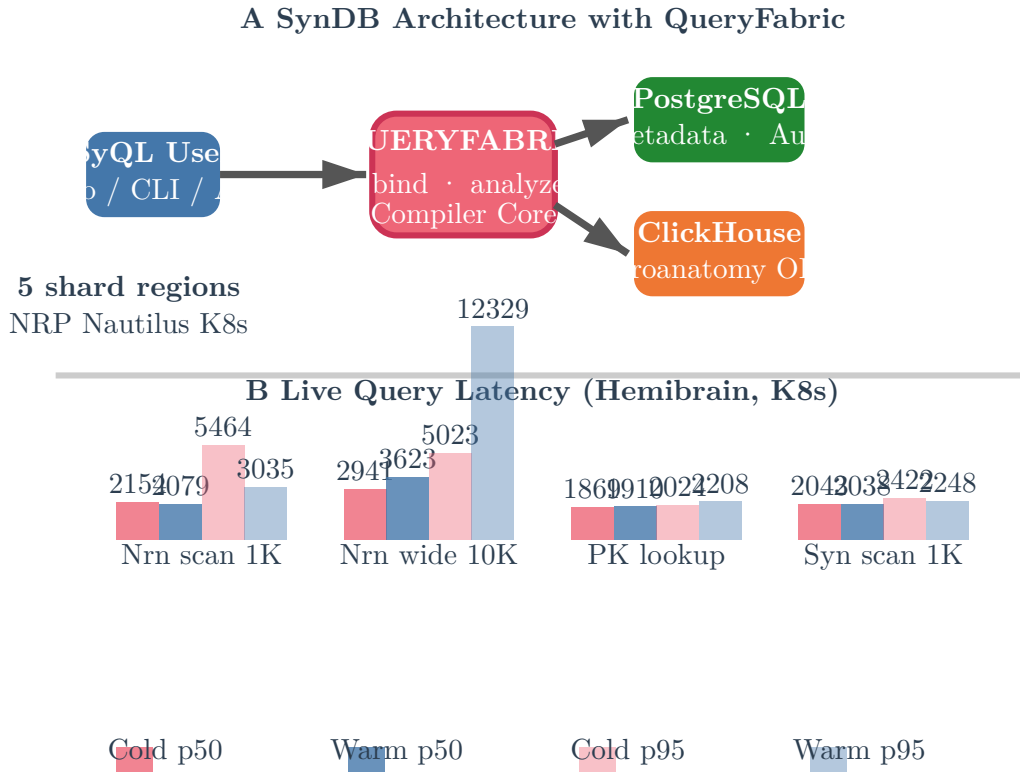


Figure 4: **SynDB case study.** (A) Simplified architecture: SynDB uses QueryFabric as the compiler layer between its SyQL dialect and ClickHouse/PostgreSQL backends, across 5 shard regions on NRP Nautilus. (B) Live query latency summary from the Hemibrain production cluster: p50 and p95 latencies for four query templates under cold and warm cache states.